

1. Introduction to FastAPI

Explanation

FastAPI is a modern, high-performance Python web framework used for building APIs efficiently. It was created by Sebastián Ramírez and is based on ASGI (Asynchronous Server Gateway Interface). FastAPI combines the performance of Node.js and Go with the simplicity of Python.

It is built on two important libraries:

- Starlette → handles web-related functionality
- Pydantic → handles validation and serialization

FastAPI automatically validates requests using Python type hints and generates interactive API documentation automatically.

FastAPI is widely used for:

- REST API development
 - AI/ML backend systems
 - Microservices
 - Real-time systems
 - Cloud-native applications
-

Key Features

- Very high performance
 - Automatic Swagger documentation
 - Automatic request validation
 - Supports asynchronous programming
 - Easy integration with databases
 - Type hint support
 - Clean code structure
-

Advantages

- Faster development
- Less boilerplate code
- Better readability
- Automatic serialization

- Better debugging support
-

Architecture

None

Client Request

→ FastAPI

→ Validation

→ Business Logic

→ Database

→ Response

Example

None

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
@app.get("/")
```

```
def home():
```

```
    return {"message": "FastAPI Running"}
```

Hands-on (How to do)

Step 1: Install FastAPI

None

```
pip install fastapi uvicorn
```

Step 2: Create main.py

None

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
@app.get("/")
def home():
    return {"message": "API Working"}
```

Step 3: Run Application

```
None
uvicorn main:app --reload
```

Step 4: Open Browser

```
None
http://127.0.0.1:8000/docs
```

2. Models in FastAPI

Explanation

Models define the structure of data used in an application. They ensure consistency between:

- database
- request body
- response body

FastAPI mainly uses:

- Pydantic models
 - SQLAlchemy models
-

Types of Models

Pydantic Models

Used for:

- validation
- serialization
- request/response handling

SQLAlchemy Models

Used for:

- database tables
 - ORM mapping
-

Why Models are Important

- Prevent invalid data
 - Maintain structure
 - Improve readability
 - Simplify API development
-

Example

```
None
from pydantic import BaseModel

class User(BaseModel):
    name: str
    age: int
```

Hands-on

Step 1: Create Pydantic Model

```
None
from pydantic import BaseModel

class Student(BaseModel):
    name: str
    marks: int
```

Step 2: Use Model in API

None

```
@app.post("/student")
def add_student(student: Student):
    return student
```

3. Pydantic

Explanation

Pydantic is a Python library used for:

- data validation
- parsing
- serialization
- enforcing type safety

FastAPI internally uses Pydantic for validating incoming requests automatically.

Features

- Automatic type checking
 - Validation errors
 - Optional fields
 - Nested models
 - Default values
-

Validation Process

None

```
Client Request
→ Pydantic Validation
→ Valid Data
→ API Logic
```

Example

None

```
from pydantic import BaseModel
```

```
class Product(BaseModel):  
    name: str  
    price: float
```

Invalid Example

None

```
{  
    "name": "Laptop",  
    "price": "abc"  
}
```

Output:

None

Validation Error

Hands-on

Optional Fields

None

```
from typing import Optional
```

```
class User(BaseModel):  
    name: str  
    age: Optional[int] = None
```

Nested Models

None

```
class Address(BaseModel):  
    city: str
```

```
class User(BaseModel):  
    name: str  
    address: Address
```

4. Schema

Explanation

Schemas define the structure of:

- request data
- response data

Schemas improve:

- validation
- consistency
- documentation
- maintainability

Schemas are generally created using Pydantic.

Types

- Request Schema
 - Response Schema
-

Benefits

- Clear API structure
 - Better frontend-backend communication
 - Automatic validation
-

Example

None

```
class ProductSchema(BaseModel):  
    name: str  
    price: float
```

Hands-on

Request Schema

None

```
@app.post("/product")  
def create_product(product: ProductSchema):  
    return product
```

Response Schema

None

```
@app.get("/product", response_model=ProductSchema)  
def get_product():  
    return {  
        "name": "Laptop",  
        "price": 50000  
    }
```

5. Path Parameters

Explanation

Path parameters are dynamic values passed inside the URL path.

They are mainly used for:

- fetching specific records
 - updating records
 - deleting records
-

Syntax

```
None  
/user/{id}
```

Advantages

- Dynamic routing
 - Easy resource identification
 - Automatic validation
-

Example

```
None  
@app.get("/user/{id}")  
def get_user(id: int):  
    return {"id": id}
```

Hands-on

Multiple Path Parameters

```
None  
@app.get("/user/{id}/orders/{order_id}")  
def get_order(id: int, order_id: int):  
    return {  
        "user_id": id,  
        "order_id": order_id  
    }
```

6. Query Parameters

Explanation

Query parameters are values passed after ? in a URL.

Used for:

- filtering
 - sorting
 - searching
 - pagination
-

Example URL

None

```
/products?category=electronics
```

Benefits

- Flexible filtering
 - Optional inputs
 - Dynamic searches
-

Example

None

```
@app.get("/products")
def get_products(category: str):
    return {"category": category}
```

Hands-on

Multiple Query Parameters

None

```
@app.get("/products")
def get_products(category: str, limit: int = 10):
    return {
        "category": category,
        "limit": limit
    }
```

7. Database Connection

Explanation

Database connection enables FastAPI applications to communicate with relational databases.

Supported databases:

- SQLite
- MySQL
- PostgreSQL

Usually implemented using:

- SQLAlchemy ORM

Architecture

None

FastAPI

→ SQLAlchemy

→ Database Engine

→ Database

Why SQLAlchemy

- ORM support
- Easier queries
- Better abstraction
- Database independence

Hands-on

Install SQLAlchemy

None

```
pip install sqlalchemy
```

Create Engine

None

```
from sqlalchemy import create_engine
```

```
DATABASE_URL = "sqlite:///./test.db"
```

```
engine = create_engine(DATABASE_URL)
```

Create Session

None

```
from sqlalchemy.orm import sessionmaker
```

```
SessionLocal = sessionmaker(bind=engine)
```

8. CRUD Operations

8.1 Create Operation

Explanation

Used to insert new records into database.

Example

None

```
@app.post("/users")
def create_user(user: User):
    return user
```

8.2 Read Operation

Explanation

Used to retrieve records from database.

Example

```
None
@app.get("/users")
def get_users():
    return {"users": []}
```

8.3 Update Operation

Explanation

Used to modify existing records.

Example

```
None
@app.put("/users/{id}")
def update_user(id: int):
    return {"message": "Updated"}
```

8.4 Delete Operation

Explanation

Used to remove records from database.

Example

None

```
@app.delete("/users/{id}")
def delete_user(id: int):
    return {"message": "Deleted"}
```

9. Alembic (Database Migration)

Explanation

Alembic is a database migration tool used with SQLAlchemy.

Migration means:

- tracking database changes
 - applying schema updates
 - version controlling database structure
-

Why Alembic is Needed

Without Alembic:

- manual schema changes
- inconsistent database state

With Alembic:

- automatic migrations
 - rollback support
 - easier collaboration
-

Alembic Workflow

None

```
Model Change
→ Generate Migration
→ Apply Migration
→ Update Database
```

Hands-on

Install Alembic

None
pip install alembic

Initialize

None
alembic init alembic

Configure Database

Inside:

None
alembic.ini

Set:

None
sqlalchemy.url = sqlite:///./test.db

Create Migration

None
alembic revision --autogenerate -m "create users table"

Apply Migration

None
alembic upgrade head

Rollback Migration

None

alembic downgrade -1

10. Swagger Documentation

Explanation

FastAPI automatically generates interactive API documentation.

Two interfaces:

- Swagger UI
 - ReDoc
-

Benefits

- Easy API testing
 - Better debugging
 - Automatic request validation
-

Access URLs

None

/docs

/redoc

11. Dependency Injection

Explanation

Dependency Injection allows reusable components such as:

- database sessions
- authentication
- services

FastAPI automatically injects dependencies when needed.

Example

```
None
from fastapi import Depends

def get_db():
    db = SessionLocal()
    yield db
    db.close()

@app.get("/")
def home(db=Depends(get_db)):
    return {"message": "Connected"}
```

12. Async Support

Explanation

FastAPI supports asynchronous APIs using:

```
None
async def
```

This improves:

- concurrency
 - scalability
 - performance
-

Example

None

```
@app.get("/")
async def home():
    return {"message": "Async API"}
```

13. Recommended Project Structure

None

```
project/
|
|— main.py
|— database.py
|— models.py
|— schemas.py
|— crud.py
|— routes/
|— alembic/
```

14. End-to-End Flow

None

```
Client Request
→ FastAPI Route
→ Pydantic Validation
→ Schema
→ Database Session
→ CRUD Logic
→ Database
→ Response
```